

| NTA Setup Backend Development Resources

| Network Interface Management Commands

So network Interface will have these options:

1. Check Available Interface
2. Interface up/down
3. Promisc mode on/off
4. check with tcpdump if getting Network Logs
5. Check with tcpdump if getting firewall logs
6. Change machine IP with Netplan Config

| 1. List and Identify Available Interfaces

```
# List all network interfaces (basic)
ip link show
# or
ifconfig -a

# List only physical interfaces (excluding virtual/loopback)
ls /sys/class/net/ | grep -v lo

# Show interface details with MAC addresses
ip link show | grep -E "^[0-9]+:" | cut -d: -f2 | tr -d ' ' | while read int; do
    mac=$(cat /sys/class/net/$int/address 2>/dev/null)
    echo "$int - MAC: $mac"
done

# List interfaces with types (physical, virtual, bridge, bond)
for interface in /sys/class/net/*; do
```

```

int=$(basename $interface)
type=$(cat $interface/type 2>/dev/null)
case $type in
    1) type="Ethernet" ;;
    2) type="Loopback" ;;
    3) type="Bridge" ;;
    *) type="Unknown ($type)" ;;
esac
echo "$int: $type"
done

# Get interface details including driver information
for interface in $(ls /sys/class/net/ | grep -v lo); do
    driver=$(basename $(readlink /sys/class/net/$interface/device/driver 2>/dev/null) 2>/dev/null)
    if [ -n "$driver" ]; then
        echo "$interface: Driver - $driver"
    else
        echo "$interface: Driver - Unknown/virtual"
    fi
done

# Comprehensive interface listing with details
printf "%-15s %-10s %-10s %-18s %s\n" "INTERFACE" "STATE" "PROMISC" "MAC" "IP ADDRESS"
for interface in $(ls /sys/class/net/ | sort); do
    state=$(cat /sys/class/net/$interface/operstate 2>/dev/null)
    promisc=$(cat /sys/class/net/$interface/flags 2>/dev/null | awk '{if($1 && 0x100) print "ON"; else print "OFF"}')
    mac=$(cat /sys/class/net/$interface/address 2>/dev/null)
    ip=$(ip -4 addr show $interface 2>/dev/null | grep inet | awk '{print $2}' | head -1)
    [ -z "$ip" ] && ip="-"
    printf "%-15s %-10s %-10s %-18s %s\n" "$interface" "$state" "$promisc" "$mac" "$ip"
done

```

I 2. Check Interface State (Up/Down)

```
# Check if specific interface is up
interface="eth0"
if ip link show $interface | grep -q "state UP"; then
    echo "$interface is UP"
else
    echo "$interface is DOWN"
fi

# Alternative method using operstate
state=$(cat /sys/class/net/$interface/operstate 2>/dev/null)
case $state in
    up) echo "$interface is UP and running" ;;
    down) echo "$interface is DOWN" ;;
    unknown) echo "$interface state is UNKNOWN" ;;
    *) echo "$interface state: $state" ;;
esac

# Check carrier signal (physical connection)
if [ "$(cat /sys/class/net/$interface/carrier 2>/dev/null)" = "1" ]; then
    echo "$interface has physical connection"
else
    echo "$interface has NO physical connection"
fi

# Get link speed and duplex if up
if [ "$(cat /sys/class/net/$interface/operstate 2>/dev/null)" = "up" ]; then
    speed=$(cat /sys/class/net/$interface/speed 2>/dev/null)
    duplex=$(cat /sys/class/net/$interface/duplex 2>/dev/null)
    echo "$interface: ${speed}Mbps, $duplex duplex"
fi

# List only interfaces that are UP
echo "=== Interfaces UP ==="
for interface in $(ls /sys/class/net/); do
```

```

    if [ "$(cat /sys/class/net/$interface/operstate 2>/dev/null)" = "up" ]; then
        ip=$(ip -4 addr show $interface 2>/dev/null | grep inet | awk '{print $2}')
        echo "$interface: $ip"
    fi
done

# List only interfaces that are DOWN
echo "=== Interfaces DOWN ==="
for interface in $(ls /sys/class/net/); do
    if [ "$(cat /sys/class/net/$interface/operstate 2>/dev/null)" = "down" ]; then
        echo "$interface"
    fi
done

# Monitor interface state changes (useful for UI real-time updates)
while true; do
    clear
    echo "=== Interface Status Monitor (Press Ctrl+C to exit) ==="
    date
    echo ""
    printf "%-15s %-10s %-10s %s\n" "INTERFACE" "STATE" "PROMISC" "IP ADDRESS"
    for interface in $(ls /sys/class/net/ | grep -v lo); do
        state=$(cat /sys/class/net/$interface/operstate 2>/dev/null)
        promisc=$(ip link show $interface | grep -o "PROMISC" || echo "no")
        ip=$(ip -4 addr show $interface 2>/dev/null | grep inet | awk '{print $2}' | head -1)
        [ -z "$ip" ] && ip="-"
        printf "%-15s %-10s %-10s %s\n" "$interface" "$state" "$promisc" "$ip"
    done
    sleep 2
done

```

3. Manage Interface State (Up/Down)

```
# Bring interface UP
sudo ip link set eth0 up
# or
sudo ifconfig eth0 up

# Bring interface DOWN
sudo ip link set eth0 down
# or
sudo ifconfig eth0 down

# Toggle interface state
interface="eth0"
if ip link show $interface | grep -q "state UP"; then
    sudo ip link set $interface down
    echo "$interface is now DOWN"
else
    sudo ip link set $interface up
    echo "$interface is now UP"
fi

# Restart interface (down then up)
sudo ip link set eth0 down && sudo ip link set eth0 up

# Restart with delay
sudo ip link set eth0 down
sleep 2
sudo ip link set eth0 up

# Safe restart (with confirmation)
interface="eth0"
if ping -c 1 -W 1 $(ip route show default | awk '/default/ {print $3}') >/dev/null 2>&1; then
    echo "Warning: This interface may be your default route"
    read -p "Continue? (y/n): " confirm
    if [ "$confirm" = "y" ]; then
```

```
        sudo ip link set $interface down && sudo ip link set $interface up
    fi
else
    sudo ip link set $interface down && sudo ip link set $interface up
fi
```

4. Promiscuous Mode Management (Persistence)

```
# Check if interface is in promiscuous mode
interface="eth0"
if ip link show $interface | grep -q "PROMISC"; then
    echo "$interface is in PROMISCUOUS mode"
else
    echo "$interface is NOT in promiscuous mode"
fi

# Alternative check using flags
flags=$(cat /sys/class/net/$interface/flags 2>/dev/null)
if [ $((flags & 0x100)) -ne 0 ]; then
    echo "$interface is in PROMISCUOUS mode"
else
    echo "$interface is NOT in promiscuous mode"
fi

# Enable promiscuous mode
sudo ip link set eth0 promisc on
# or
sudo ifconfig eth0 promisc

# Disable promiscuous mode
sudo ip link set eth0 promisc off
# or
sudo ifconfig eth0 -promisc
```

```

# Toggle promiscuous mode
interface="eth0"
if ip link show $interface | grep -q "PROMISC"; then
    sudo ip link set $interface promisc off
    echo "Promiscuous mode DISABLED on $interface"
else
    sudo ip link set $interface promisc on
    echo "Promiscuous mode ENABLED on $interface"
fi

# Enable promiscuous mode permanently (for Suricata)
# Add to /etc/network/interfaces (Debian/Ubuntu)
echo "auto eth0
iface eth0 inet manual
    up ip link set eth0 promisc on
    down ip link set eth0 promisc off" | sudo tee -a /etc/network/interfaces

# For systemd-networkd, create a network file
sudo bash -c 'cat > /etc/systemd/network/10-eth0.network << EOF
[Match]
Name=eth0

[Link]
Promiscuous=yes
EOF'

# Apply systemd-networkd changes
sudo systemctl restart systemd-networkd

```

5. Comprehensive Interface Management Script (Example)

```

#!/bin/bash
# interface_manager.sh - Complete interface management tool

```

```
INTERFACE="$1"
ACTION="$2"

show_help() {
    echo "Usage: $0 <interface> <action>"
    echo "Actions:"
    echo "  status      - Show interface status"
    echo "  up          - Bring interface up"
    echo "  down        - Bring interface down"
    echo "  restart     - Restart interface"
    echo "  promisc-on  - Enable promiscuous mode"
    echo "  promisc-off - Disable promiscuous mode"
    echo "  promisc-toggle - Toggle promiscuous mode"
    echo "  details     - Show detailed interface info"
    echo "  monitor     - Monitor interface changes"
    echo ""
    echo "Examples:"
    echo "  $0 eth0 status"
    echo "  $0 eth0 promisc-on"
}

show_status() {
    if [ ! -e "/sys/class/net/$INTERFACE" ]; then
        echo "Error: Interface $INTERFACE does not exist"
        exit 1
    fi

    echo "=== Interface: $INTERFACE ==="

    # Basic status
    state=$(cat /sys/class/net/$INTERFACE/operstate 2>/dev/null)
    echo "State: $state"

    # Promiscuous mode
    if ip link show $INTERFACE | grep -q "PROMISC"; then
```

```
    echo "Promiscuous: ENABLED"
else
    echo "Promiscuous: DISABLED"
fi

# MAC address
mac=$(cat /sys/class/net/$INTERFACE/address 2>/dev/null)
echo "MAC: $mac"

# IP addresses
echo "IP Addresses:"
ip -4 addr show $INTERFACE 2>/dev/null | grep inet | awk '{print " IPv4: " $2}'
ip -6 addr show $INTERFACE 2>/dev/null | grep inet6 | awk '{print " IPv6: " $2}'

# Speed and duplex
if [ "$state" = "up" ]; then
    speed=$(cat /sys/class/net/$INTERFACE/speed 2>/dev/null)
    duplex=$(cat /sys/class/net/$INTERFACE/duplex 2>/dev/null)
    [ -n "$speed" ] && echo "Speed: ${speed}Mbps"
    [ -n "$duplex" ] && echo "Duplex: $duplex"
fi

# Driver
driver=$(basename $(readlink /sys/class/net/$INTERFACE/device/driver 2>/dev/null) 2>/dev/null)
[ -n "$driver" ] && echo "Driver: $driver"

# Statistics
echo "Statistics:"
rx_packets=$(cat /sys/class/net/$INTERFACE/statistics/rx_packets 2>/dev/null)
tx_packets=$(cat /sys/class/net/$INTERFACE/statistics/tx_packets 2>/dev/null)
rx_bytes=$(cat /sys/class/net/$INTERFACE/statistics/rx_bytes 2>/dev/null)
tx_bytes=$(cat /sys/class/net/$INTERFACE/statistics/tx_bytes 2>/dev/null)
rx_errors=$(cat /sys/class/net/$INTERFACE/statistics/rx_errors 2>/dev/null)
tx_errors=$(cat /sys/class/net/$INTERFACE/statistics/tx_errors 2>/dev/null)
```

```

echo "  RX: $rx_packets packets, $rx_bytes bytes, $rx_errors errors"
echo "  TX: $tx_packets packets, $tx_bytes bytes, $tx_errors errors"
}

show_details() {
  # Show all available information about interface
  echo "=== Detailed Information for $INTERFACE ==="
  ethtool $INTERFACE 2>/dev/null || echo "ethtool not available or interface doesn't support"
  echo ""
  echo "=== System Info ==="
  udevadm info -a -p /sys/class/net/$INTERFACE 2>/dev/null | grep -E "looking at
device|ATTRS{device}|ATTRS{vendor}" | head -10
}

monitor_interface() {
  echo "Monitoring $INTERFACE for changes (Ctrl+C to stop)"
  echo "Timestamp, State, Promisc, RX_packets, TX_packets, RX_bytes, TX_bytes"

  old_state=""
  old_promisc=""

  while true; do
    timestamp=$(date +%s)
    state=$(cat /sys/class/net/$INTERFACE/operstate 2>/dev/null)
    promisc=$(ip link show $INTERFACE | grep -q "PROMISC" && echo "1" || echo "0")
    rx_packets=$(cat /sys/class/net/$INTERFACE/statistics/rx_packets 2>/dev/null)
    tx_packets=$(cat /sys/class/net/$INTERFACE/statistics/tx_packets 2>/dev/null)
    rx_bytes=$(cat /sys/class/net/$INTERFACE/statistics/rx_bytes 2>/dev/null)
    tx_bytes=$(cat /sys/class/net/$INTERFACE/statistics/tx_bytes 2>/dev/null)

    echo "$timestamp, $state, $promisc, $rx_packets, $tx_packets, $rx_bytes, $tx_bytes"

    if [ "$state" != "$old_state" ]; then
      echo "ALERT: Interface state changed to $state at $(date)"
    fi
  fi
}

```

```

        if [ "$promisc" != "$old_promisc" ]; then
            echo "ALERT: Promiscuous mode changed to $( [ "$promisc" = "1" ] && echo "ON" || echo "OFF") at
$(date)"
        fi

        old_state=$state
        old_promisc=$promisc
        sleep 2
    done
}

# Main execution
case $ACTION in
    status)
        show_status
        ;;
    up)
        sudo ip link set $INTERFACE up
        echo "$INTERFACE is now UP"
        ;;
    down)
        sudo ip link set $INTERFACE down
        echo "$INTERFACE is now DOWN"
        ;;
    restart)
        sudo ip link set $INTERFACE down
        sleep 2
        sudo ip link set $INTERFACE up
        echo "$INTERFACE restarted"
        ;;
    promisc-on)
        sudo ip link set $INTERFACE promisc on
        echo "Promiscuous mode ENABLED on $INTERFACE"
        ;;
    promisc-off)

```

```

        sudo ip link set $INTERFACE promisc off
        echo "Promiscuous mode DISABLED on $INTERFACE"
    ;;
promisc-toggle)
    if ip link show $INTERFACE | grep -q "PROMISC"; then
        sudo ip link set $INTERFACE promisc off
        echo "Promiscuous mode DISABLED on $INTERFACE"
    else
        sudo ip link set $INTERFACE promisc on
        echo "Promiscuous mode ENABLED on $INTERFACE"
    fi
    ;;
details)
    show_details
    ;;
monitor)
    monitor_interface
    ;;
list)
    echo "=== Available Interfaces ==="
    printf "%-15s %-10s %-10s %-18s %s\n" "INTERFACE" "STATE" "PROMISC" "MAC" "IP"
    for int in $(ls /sys/class/net/ | sort); do
        state=$(cat /sys/class/net/$int/operstate 2>/dev/null)
        promisc=$(ip link show $int | grep -q "PROMISC" && echo "ON" || echo "OFF")
        mac=$(cat /sys/class/net/$int/address 2>/dev/null)
        ip=$(ip -4 addr show $int 2>/dev/null | grep inet | awk '{print $2}' | head -1)
        [ -z "$ip" ] && ip="-"
        printf "%-15s %-10s %-10s %-18s %s\n" "$int" "$state" "$promisc" "$mac" "$ip"
    done
    ;;
*)
    show_help
    ;;
esac

```

6. Example Script: JSON Output for UI Integration

```
#!/bin/bash
# interface_json.sh - Output interface info in JSON format for UI

interfaces_json() {
    echo "{"
    echo "  \"interfaces\": ["
    first=1
    for interface in $(ls /sys/class/net/ | sort); do
        if [ $first -eq 0 ]; then
            echo ","
        fi
        first=0

        state=$(cat /sys/class/net/$interface/operstate 2>/dev/null)
        promisc=$(ip link show $interface | grep -q "PROMISC" && echo "true" || echo "false")
        mac=$(cat /sys/class/net/$interface/address 2>/dev/null)
        ipv4=$(ip -4 addr show $interface 2>/dev/null | grep inet | awk '{print $2}' | head -1)
        ipv6=$(ip -6 addr show $interface 2>/dev/null | grep inet6 | awk '{print $2}' | head -1)
        speed=$(cat /sys/class/net/$interface/speed 2>/dev/null)
        duplex=$(cat /sys/class/net/$interface/duplex 2>/dev/null)
        mtu=$(cat /sys/class/net/$interface/mtu 2>/dev/null)

        cat <<EOF
{
    "name": "$interface",
    "state": "$state",
    "promiscuous": $promisc,
    "mac": "$mac",
    "ipv4": "${ipv4:-null}",
    "ipv6": "${ipv6:-null}",
    "speed": ${speed:-null},
    "duplex": "${duplex:-null}",

```

```

        "mtu": ${mtu:-null}
    }
EOF
done
echo "  ]"
echo "}"
}

# Single interface JSON
interface_json() {
    interface=$1
    if [ ! -e "/sys/class/net/$interface" ]; then
        echo '{"error": "Interface not found"}'
        exit 1
    fi

    state=$(cat /sys/class/net/$interface/operstate 2>/dev/null)
    promisc=$(ip link show $interface | grep -q "PROMISC" && echo "true" || echo "false")
    mac=$(cat /sys/class/net/$interface/address 2>/dev/null)
    ipv4=$(ip -4 addr show $interface 2>/dev/null | grep inet | awk '{print $2}' | head -1)
    ipv6=$(ip -6 addr show $interface 2>/dev/null | grep inet6 | awk '{print $2}' | head -1)
    speed=$(cat /sys/class/net/$interface/speed 2>/dev/null)
    duplex=$(cat /sys/class/net/$interface/duplex 2>/dev/null)
    mtu=$(cat /sys/class/net/$interface/mtu 2>/dev/null)
    driver=$(basename $(readlink /sys/class/net/$interface/device/driver 2>/dev/null) 2>/dev/null)

    cat <<EOF
{
    "name": "$interface",
    "state": "$state",
    "promiscuous": $promisc,
    "mac": "$mac",
    "ipv4": "${ipv4:-null}",
    "ipv6": "${ipv6:-null}",
    "speed": ${speed:-null},

```

```

"duplex": "${duplex:-null}",
"mtu": ${mtu:-null},
"driver": "${driver:-null}",
"statistics": {
    "rx_packets": $(cat /sys/class/net/$interface/statistics/rx_packets 2>/dev/null || echo 0),
    "tx_packets": $(cat /sys/class/net/$interface/statistics/tx_packets 2>/dev/null || echo 0),
    "rx_bytes": $(cat /sys/class/net/$interface/statistics/rx_bytes 2>/dev/null || echo 0),
    "tx_bytes": $(cat /sys/class/net/$interface/statistics/tx_bytes 2>/dev/null || echo 0),
    "rx_errors": $(cat /sys/class/net/$interface/statistics/rx_errors 2>/dev/null || echo 0),
    "tx_errors": $(cat /sys/class/net/$interface/statistics/tx_errors 2>/dev/null || echo 0)
}
}
EOF
}

# Usage
case "$1" in
    list)
        interfaces_json
        ;;
    get)
        interface_json "$2"
        ;;
    *)
        echo "Usage: $0 {list|get <interface>}"
        exit 1
        ;;
esac

```

6. Network Interface Configuration

```

# List all network interfaces
ip link show
ifconfig -a

```

```
# Get interface details
ip addr show <interface-name>
ethtool <interface-name>

# Set interface up/down
sudo ip link set <interface-name> up
sudo ip link set <interface-name> down

# Get interface statistics
ip -s link show <interface-name>
netstat -i

# Configure interface for promiscuous mode (for Suricata)
sudo ip link set <interface-name> promisc on

# Check if interface is in promiscuous mode
ip link show <interface-name> | grep PROMISC

# Get default gateway
ip route show default

# TCPDUMP forwarded log checks (for networks)
tcpdump -i <span-interface>
```

Example Netplan Config:

User has the right to change in every value. Also can add or remove interface, All will be done via UI, We wont show user this following format, we just take every value based on the configuration variable:

```
network:
  version: 2
  ethernets:
    enp2s0:
```

```
addresses:
  - "10.10.4.22/23"      # Management interface
nameservers:
  addresses:
    - "10.1.2.16"
  search:
    - "amplifysec.net"
routes:
  - to: "default"
    via: "10.10.5.254"

enp3s0:                  # Monitoring interface (SPAN)
  dhcp4: no
  dhcp6: no
  optional: true

enp4s0:
  addresses:
    - "10.118.118.222/24"
  dhcp4: no
  dhcp6: no
  optional: true
```

| System Service Management Commands Document

| 1. Service Control Commands

```
# Start stop and Restart services
sudo systemctl start suricata
sudo systemctl start logstash
sudo systemctl start wazuh-agent
sudo systemctl start rsyslog
sudo systemctl start rport
```

```
# Stop a service
sudo systemctl stop <service-name>

# Restart a service
sudo systemctl restart <service-name>

# Check service status
sudo systemctl status <service-name>

# List all services and their status
sudo systemctl list-units --type=service --all

# Check if service is active (returns 0 if active, non-zero if not)
systemctl is-active --quiet <service-name> && echo "Running" || echo "Stopped"
```

| 2. Service-Specific Commands

| Logstash (Log Shipper)

```
# Test Logstash configuration
sudo /usr/share/logstash/bin/logstash --config.test_and_exit -f /etc/logstash/conf.d/

# View Logstash logs
sudo journalctl -u logstash -f
sudo tail -f /var/log/logstash/logstash.log

# Check Logstash version
sudo /usr/share/logstash/bin/logstash --version

# Check running pipelines
This one is more accurate:
curl -s http://localhost:9600/_node/pipelines | jq '.pipelines | keys'

This one is Alternative
```

```
sudo grep "Pipelines running" /var/log/logstash/logstash-plain.log | tail -1
```

Unable Disable a Pipeline

```
sudo sed -i '/pipeline.id: suricata-alerts/,/queue.type/ s/^#/' /etc/logstash/pipelines.yml
```

Enable pipeline (uncomment)

```
sudo sed -i '/pipeline.id: suricata-alerts/,/queue.type/ s/^#/' /etc/logstash/pipelines.yml
```

Available Pipeline:

1. network
2. edr
3. asset-discovery
4. identities-access

| So Logstash will have four option:

1. Service Status
2. Service Start
3. Service Stop
4. Service Restart
5. Syntax check/Config check
6. Check Running Pipelines
7. Unable/Disable Pipelines

| Suricata (Network Threat Analysis - NTA)

```
# Check Suricata status
```

```
sudo systemctl status suricata
```

```
# Restart Suricata
```

```
sudo systemctl restart suricata
```

```
# Test Suricata configuration
sudo suricata -T -c /etc/suricata/suricata.yaml

# Update Suricata rules
sudo suricata-update

# Set home network in Suricata config
sudo sed -i "s/HOME_NET:./HOME_NET: \"[192.168.1.0/24]\"/" /etc/suricata/suricata.yaml

# Check Suricata running interfaces
sudo ss -tulpn | grep suricata
sudo ps aux | grep suricata

# View Suricata alerts
sudo tail -f /var/log/suricata/eve.json | jq 'select(.event_type=="alert")'

# Check Suricata stats
sudo tail -f /var/log/suricata/stats.log

# Check current interface
grep -A2 "af-packet:" /etc/suricata/suricata.yaml | grep interface

# Basic Interface Change
sudo sed -i 's/interface: enp3s0/interface: eth0/' /etc/suricata/suricata.yaml

# Change interface with variable
NEW_IF="eth0"; sudo sed -i "s/interface: enp3s0/interface: $NEW_IF/" /etc/suricata/suricata.yaml
```

! So Suricata will have four option:

1. Service Status
2. Service Start
3. Service Stop

4. Service Reload
5. Service Restart
6. Syntax check
7. Alerts Check
8. Check Running Interface
9. Set Home net
10. Change Interface
11. Service Update

| OpenRPort (Remote Customer Service)

```
# Check RPort status
sudo systemctl status rport
sudo systemctl restart rport
```

Check **if** Connected:

```
sudo journalctl -u rport -n 20 | grep "Connected to"
```

-n 20 for last 20 lines, can increase this and **sort** based on ascending order also.

| So Logstash will have four option:

1. Service Status
2. Service Start
3. Service Stop
4. Service Restart
5. Check latest connection status

| Rsyslog (For Firewall Logs)

```
# Enable (uncomment) firewall config
```

```
sudo sed -i 's/^#\(\input(type="imudp" port="514")\)/\1/' /etc/rsyslog.d/01-firewall.conf && sudo sed -i  
's/^#\(\(if \$inputname == "imudp" then {\}\)/\1/; s/^#\(\( action(\)\)/\1/; s/^#\(\( type="omfile"\)/\1/; s/^#\(\(  
file="\var\log\firewall.log"\)/\1/; s/^#\(\( )\)/\1/; s/^#\(\( stop\)/\1/; s/^#\(\{\}\)/\1/'  
/etc/rsyslog.d/01-firewall.conf && sudo systemctl restart rsyslog
```

```
# Disable (comment out) firewall config
```

```
sudo sed -i 's/^#\(\input(type="imudp" port="514")\)/#\1/' /etc/rsyslog.d/01-firewall.conf && sudo sed -i  
's/^#\(\(if \$inputname == "imudp" then {\}\)/#\1/; s/^#\(\( action(\)\)/#\1/; s/^#\(\( type="omfile"\)/#\1/; s/^#\(  
file="\var\log\firewall.log"\)/#\1/; s/^#\(\( )\)/#\1/; s/^#\(\( stop\)/#\1/; s/^#\(\{\}\)/#\1/'  
/etc/rsyslog.d/01-firewall.conf && sudo systemctl restart rsyslog
```

```
# First Check if Firewall logs coming to this Machine
```

```
tcpdump -i <management-interface> -n port 514
```

```
# Check configuration status
```

```
### Check if firewall config is enabled
```

```
grep -v "^#" /etc/rsyslog.d/01-firewall.conf | grep -q "input.*port=\"514\"" && echo "Enabled" || echo  
"Disabled"
```

```
# Check if port 514 is listening
```

```
sudo ss -tulpn | grep :514
```

```
#Test rsyslog configuration
```

```
sudo rsyslogd -N1
```

```
# Check rsyslog status
```

```
sudo systemctl status rsyslog
```

```
# Tail firewall log (wait for incoming logs)
sudo tail -f /var/log/firewall.log
```

| So Rsyslog will have four option:

1. Service Status
2. Service Start
3. Service Stop
4. Service Restart
5. Service Enable/Disable
6. Check Listening port
7. Check is config is Enabled
8. Check/tail Logs

| Wazuh-Agent (Firewall Log Shipper)

```
# Check Wazuh agent status
sudo systemctl status wazuh-agent

# Restart Wazuh agent
sudo systemctl restart wazuh-agent

# Check Wazuh agent connection status
sudo /var/ossec/bin/agent-control -l

# View Wazuh agent logs
sudo tail -f /var/ossec/logs/ossec.log
sudo journalctl -u wazuh-agent -f
```

| Troubleshooting Commands

```
# Check if service is listening on correct port
```

```
netstat -tulpn | grep <service-port>
```

```
# Test connectivity to Server & Verify Firewall ports are open
```

```
telnet fusionxdr.amplifysec.net 6641
```

```
telnet fusionxdr.amplifysec.net 6642
```

```
telnet fusionxdr.amplifysec.net 9908
```

```
telnet fusionxdr.amplifysec.net 443
```

```
telnet s0.amplifysec.net 10050
```

Note:

- Successful connection: If you get connected, you will only see a cursor is blinking.
- Unsuccessful connection: Connecting To fusionxdr.amplifysec.net...